AI & Data Science on Databricks

Most large organizations already run analytics and have started AI projects.

An agent a business user talks to should read the same data, under the same rules, as the analytics do.

Databricks is one platform built in layers, and each layer rests on the one below it.

Vertical Applications	CustomerLake	Lakewatch					
Developer Tools	Omnigent	Agentic Code Converter					
AI for Business Users	Genie One	Genie Agents	Genie Ontology	Genie Integrations	AI/BI Dashboards	Applications	Genie ZeroOps	Genie Budgets
AI Governance & Security	Unity Catalog for AI Assets	Unity Gateway: Policies	Unity Gateway: Monitoring	Unity Gateway: Cost & Capacity	Unity Gateway: Coding Agent Governance	Security & Compliance		
Agent Development	Model Access	Custom LLMs	Batch Inference: AI Functions	Knowledge Retrieval	Tools	Agent Build	Agent Deployment	Agent Memory
Observability & Evaluation	Tracing with MLflow	Evaluation with MLflow	Prompt Registry	Runtime Telemetry				
Data Science & Machine Learning	Workspace	Feature Engineering in Unity Catalog	Training Compute	Model Lifecycle with MLflow	Model Serving	Deployment Automation		
Unity Catalog	Catalog & Access	Policy & Classification	Observability	Cost & Usage	Semantic Definitions	Data Sharing		
Data Storage & Movement	Table Formats	Ingestion & Pipelines	Transactional & Real-Time Data	Table Maintenance				

CHAPTER 1

Data Storage & Movement

Every model and agent reads data, and that data has to be stored somewhere first.

This layer decides where the data lives and in what format, and open formats in the organization's own storage keep every engine able to read it.

Everything built later reads from here.

- Table Formats
- Ingestion & Pipelines
- Transactional & Real-Time Data
- Table Maintenance

DATA STORAGE & MOVEMENT

1.1 Table Formats

Keep one consistent copy of your data, in an open format.

The first decision is how the data is stored, because every engine, model and agent downstream inherits it.

CHALLENGES

- Batch and streaming data often sit in separate stores, sometimes in closed or complex formats.
- A job that fails halfway can leave a table inconsistent.
- A schema change can break the jobs that read it.

Delta Lake is the default table format.

- The tables live in **your own cloud storage**.
- Delta adds **transactions** to the files in that storage.
- Batch jobs and streaming jobs write to the **same tables** without corrupting them.
- The schema can **change without breaking readers**.
- Tables can also use **Apache Iceberg**, the other open table format.
- **Multi-table transactions** commit changes to several tables at once.

Delta Lake GA

Delta Lake is the default table format, with ACID transactions on files in object storage for batch and streaming writers.

Data **corruption** mid-stream	**Consistency**	**ACID** transactions
Enforcement **breaks** on change	**Schema**	**Evolution** supported
Separate stores for batch/stream	**Workloads**	**Unified** ingestion layer
Proprietary or complex formats	**Format**	**Open** Delta standard
Transactional **inconsistency**	**Reliability**	Consistency **guaranteed**

lakehouse ↗

Apache Iceberg GA

Apache Iceberg tables are managed by Unity Catalog with the same transactions and permissions as Delta tables.

DATA STORAGE & MOVEMENT

1.2 Ingestion & Pipelines

Source data lands in the tables with no pipeline code to write or run.

Open tables are only useful once the data from the source systems reaches them.

CHALLENGES

- Getting it there means writing connector code for each source and wiring job dependencies by hand.

Lakeflow covers ingestion and transformation.

- Lakeflow Connect provides **managed connectors** to databases and SaaS applications, query-based and community connectors, and Auto Loader for files.
- Zerobus Ingest brings in **event data**.
- Spark Declarative Pipelines **describe** the transformations, and Lakeflow works out the **order** in which to run them.
- They are written in the Lakeflow Pipelines Editor, or on a **visual canvas** in Lakeflow Designer, and can run in **real-time mode**.
- Lakeflow Jobs **orchestrates** the workflows.

Lakeflow Connect *ingestion connectors* GA

Lakeflow Connect provides connectors to databases and SaaS applications with change data capture.

- Managed connectors
- Query-based connectors
- Community connectors BETA
- Auto Loader *file ingestion*

Spark Declarative Pipelines *pipeline framework* GA

Spark Declarative Pipelines describe transformations and dependencies; the engine computes the execution order.

- Lakeflow Pipelines Editor *authoring UI*
- Real-time mode PUBLIC PREVIEW

Lakeflow Designer *visual pipeline builder* GA

Lakeflow Designer is a visual canvas for building pipelines without code.

DATA STORAGE & MOVEMENT

1.3 Transactional & Real-Time Data

Applications and agents keep their data on the same platform as the analytics, under the same governance.

Analytical tables are in place. Applications and agents also produce and read operational data, record by record, which is a different kind of workload.

CHALLENGES

- That data usually lives in a separate database with its own copy, and it is not governed there.
- Feeding analytics from it takes replication pipelines and a second system to maintain.
- The copies lag behind the source.

Lakebase keeps that data next to the analytics, under the same rules.

- It is a **serverless Postgres** database inside the platform, for the **state of applications and agents**.
- Synced tables bring lakehouse data **into Lakebase**, and Lakehouse Sync writes Lakebase tables **back to the lakehouse**.
- Lakehouse//RT answers SQL queries **in under a second** directly on lakehouse tables, so dashboards and applications read them **without a separate serving copy**.
- LTAP is the architecture these capabilities deliver: **one dataset**, written in Lakebase and read in Lakehouse//RT, with **no replication pipeline** between them.

Lakebase *transactional database, LTAP architecture* GA

Lakebase is a serverless Postgres database inside the platform for application and agent state; the LTAP architecture joins it to the lakehouse without a replication pipeline.

- Synced tables *lakehouse to Lakebase*
- Lakehouse Sync *Lakebase to lakehouse* PUBLIC PREVIEW

Session memory **external** to DB	**Agent state**	**Native** long/short-term store
Online/offline **mismatch**	**Features**	**Low-latency** serving
App data **ungoverned**	**Governance**	**UC-synced** tables
Prod data **in** test env	**Dev workflow**	Git-like **branches**
Single-point **failure**	**Reliability**	**High availability** (99.99% SLA)

oltp projects ↗ press release ↗ oltp ↗ ltap overview ↗

DATA STORAGE & MOVEMENT

1.4 Table Maintenance

Tables stay fast on their own, with no maintenance jobs to schedule or own.

Tables written all day, by pipelines and by applications, fill with small files and old versions, and query times suffer.

- **Predictive Optimization** is enabled by default on Unity Catalog managed tables.
- It decides when to **compact files, remove old versions and cluster the data**, based on how it is queried.
- It runs on **serverless compute**, and each decision is **recorded for audit**.

Predictive Optimization GA

Predictive Optimization decides when to compact files, remove old versions and cluster data, based on how tables are queried.

Manual OPTIMIZE scheduling	**Maintenance**	**ROI-driven** auto-running
Clustering keys **hand-tuned**	**Clustering**	**Automatic** key selection
Maintenance **overtunes** tables	**Efficiency**	**Query-aware** decisions
Only **UC managed** eligible	**Scope**	**Universal** UC support
Maintenance job **overhead**	**Operations**	**Serverless** compute

predictive optimization ↗

*The data is stored, fed and maintained. The next question is **who may read it**, under which rules, and whether it can be **trusted**. All of that is decided in **one place**, Unity Catalog, for every consumer that comes after.*

CHAPTER 2

Unity Catalog

Every table, model and agent on the platform is registered in Unity Catalog.

A rule set there once applies to every tool and every user, which is what makes AI on company data governable.

For a decision-maker, this layer is where the risk of AI on company data is managed.

- Catalog & Access
- Policy & Classification
- Observability
- Cost & Usage
- Semantic Definitions
- Data Sharing

UNITY CATALOG

2.1 Catalog & Access

One set of permissions and one lineage graph cover every asset on the platform.

Registration comes first: an asset the catalog does not know cannot be governed.

- The catalog organizes **every asset**, and Discover lets users **find** them.
- **Lineage** follows the data across the platform, and external lineage extends it to **systems outside Databricks**.
- Access is granted **by role or by attribute**.
- Users can **request access** to an asset.
- **Other engines** access the same tables through catalog commits and the Unity REST API.
- **Secrets** are stored and governed in the catalog.

Catalog & Access GA

Unity Catalog holds every asset and applies one set of permissions, lineage and audit to all of them.

- Catalog
- Discover *asset discovery*
- Lineage
- Role-based access control
- Attribute-based access control
- Access requests PUBLIC PREVIEW
- External engine access
- Secrets

unity catalog ↗ discover page ↗

UNITY CATALOG

2.2 Policy & Classification

A rule written once follows sensitive data wherever it is copied.

Permissions by role or attribute say who may read a table. They do not say what the table holds, and a rule about personal data needs to know that.

- Data classification **tags sensitive data** such as PII.
- Governed tags carry those labels, and tag propagation applies them to the **data derived** from a tagged table.
- The tags feed **row- and column-level access policies**.
- Governance Hub shows the state of governance in **one dashboard**.

Policy & Classification GA

Classification labels sensitive data, and policies attach to the labels.

- Data classification
- Governed tags
- Governance Hub *governance dashboard* BETA

unity catalog ↗

UNITY CATALOG

2.3 Observability

Quality problems in tables are detected before users notice them.

Access and policies are set. The data they cover also has to stay fresh and complete, or every model and answer downstream inherits the fault.

CHALLENGES

- Data freshness is invisible and nobody is notified, so a stale table can go unnoticed.
- Repairing it is manual and risky.

Data Quality Monitoring puts quality checks where access is controlled.

- Profiles track metrics over time and detect **drift** in tables and in predictions.
- Anomaly detection flags **stale or broken tables** and can trigger a **repair job**.
- Health indicators show the **quality and freshness** of each table in the catalog.
- Alerts go out by **email, Slack or webhook**.

Data Quality Monitoring GA

Data Quality Monitoring tracks freshness, completeness and drift on tables and predictions.

- Anomaly detection PUBLIC PREVIEW
- Health indicators PUBLIC PREVIEW

data profiling ↗ unity catalog ↗ data quality monitoring ↗

UNITY CATALOG

2.4 Cost & Usage

Know what each workload costs.

Every job, query and endpoint that reads this data runs on metered compute.

- System tables hold the platform's **billing and usage data**.
- They are queried **with SQL** like any other table.
- **Audit logs** and the history of automatic table maintenance are recorded there too.

Cost & Usage GA

Usage and billing data of the platform are queryable tables.

- System tables *billing and usage data*

billing ↗

UNITY CATALOG

2.5 Semantic Definitions

A metric is computed the same way by every tool.

A readable, trusted table can still be misread when "revenue" or "active customer" means one thing in the report and another in the query.

- Metric views define **governed, reusable metrics** in Unity Catalog, computed at query time.
- Domains group assets **by business domain**.
- Pages describe **business concepts**.
- Genie, dashboards and agents **read these definitions** (see AI for Business Users).

Metrics *metric views* GA

Metric views define governed, reusable metrics computed at query time.

metric views ↗

UNITY CATALOG

2.6 Data Sharing

Data leaves the organization under the same governance as inside it.

Not every consumer is inside the organization. Partners, customers and providers receive data too, and the rules should not stop at the firewall.

- OpenSharing is the **protocol** for sharing data.
- SecureConnect shares data that sits **behind a firewall**.
- Clean Rooms let organizations work on shared data **without exposing their raw data** to each other.
- Marketplace lists data, models and applications **from other providers**.

OpenSharing *sharing protocol* GA

OpenSharing is the open protocol for sharing tables, views and other assets across organizations and platforms.

- SecureConnect *sharing behind a firewall*

opensharing ↗

Clean Rooms GA

Clean Rooms let organizations compute on shared data without exposing their raw data to each other.

*With the data stored and governed, the platform can be put to work. The first users are **data scientists**, who build models on these tables with the **permissions they already hold**, and whose features, experiments and models land in the **same catalog**.*

CHAPTER 3

Data Science & Machine Learning

Classical machine learning is where most organizations already have results, and where the governed data first turns into predictions.

This layer decides whether a prediction can be reproduced, traced to its data and put in front of an application.

Each step reuses the catalog, and each produces assets that agents will reuse.

- Workspace
- Feature Engineering in Unity Catalog
- Training Compute
- Model Lifecycle with MLflow
- Model Serving
- Deployment Automation

DATA SCIENCE & MACHINE LEARNING

3.1 Workspace

Data scientists work on governed tables with a coding assistant in the same place.

Work starts where the data is.

- Data scientists work in notebooks that run **Python, SQL, R and Scala** directly on the governed tables.
- Several people can **edit the same notebook at once**, and notebooks are **versioned in Git**.
- Exploration uses the **tools the team already knows**: pandas, Spark, SQL and built-in data profiles.
- Genie Code is the **AI assistant** inside notebooks, the SQL editor, pipelines and dashboards.
- It **writes and explains code** from plain-language requests, in a pane next to the notebook or on a full page.
- A **command-line version** brings it into the terminal and into CI pipelines.
- Genie Code also runs **scheduled tasks**, including as a task in Lakeflow Jobs.

Notebooks GA

Notebooks run Python, SQL, R and Scala with real-time collaboration and Git versioning.

notebooks ↗ notebooks collaborate ↗ genie code ↗ repos ↗

Genie Code *coding assistant* GA

Genie Code is the coding assistant of the workspace, in notebooks, the SQL editor, pipelines and dashboards.

- Genie Code pane *in notebooks*
- Full-page Genie Code
- Genie Code CLI BETA
- Scheduled tasks
- Lakeflow Jobs task BETA

Writing SQL and pipelines slow and repetitive	**Speed**	Genie Code generates SQL and code from natural language prompts

DATA SCIENCE & MACHINE LEARNING

3.2 Feature Engineering in Unity Catalog

Build features once and reuse them across teams and models.

Exploration ends in features, the inputs a model learns from, and a feature is only worth keeping if the next team can find it and trust it.

CHALLENGES

- A feature computed one way for training and another way in production gives wrong predictions.
- Teams duplicate the same features across projects.
- Nobody sees where each one comes from or who may use it.

Feature Engineering in Unity Catalog ends the duplication with one shared registry.

- Feature definitions carry the **same permissions and lineage** as the tables they come from.
- Teams write features to feature tables, or declare them as Feature Views, which **compute them automatically**.
- Stream Feature Views compute them from **streaming data**.
- Training sets are built with **point-in-time correct joins**, and the **same definitions feed predictions** in production.
- For real-time use, features are served from an **online store on Lakebase**.
- Feature serving returns them **over REST**, and models **look them up automatically** at inference time.

Feature Engineering in Unity Catalog GA

Features are defined once in Unity Catalog and reused for training and serving.

- Feature tables
- Feature Views
- Online Feature Stores
- Feature serving

Features computed differently at training vs inference cause **skew** in production	**Training/serving**	**Unified** feature computation ensures consistency across pipeline
Teams **duplicate** features across projects wasting resources	**Feature management**	**Shared** feature registry enables centralized reuse
No visibility into feature **origins** and dependencies	**Lineage**	Automatic **lineage** tracking connects features to data sources
Ungoverned **access** to features across	**Governance**	Unity Catalog integration provides **access**

DATA SCIENCE & MACHINE LEARNING

3.3 Training Compute

Teams train large models without building or operating clusters.

Training on those features is the first step where compute becomes the constraint: some datasets and models do not fit on one machine.

CHALLENGES

- Distributed training code is hard to write correctly.
- GPUs are scarce.
- Clusters configured by hand are error-prone or sit idle.

Databricks Runtime for Machine Learning and **AI Runtime** are the two compute options.

- Databricks Runtime for Machine Learning runs training on **classic clusters**, with managed libraries such as TorchDistributor, DeepSpeed and Ray.
- AI Runtime provides **GPUs on demand** with no cluster to manage, and it **scales to zero** when the job ends.
- The AI Runtime SDK **spreads training across GPUs**, and the AI Runtime CLI submits it from the command line.
- **Fine-tuning** of language models runs on the same AI Runtime (see Agent Development).

Databricks Runtime for Machine Learning *classic compute* GA

Databricks Runtime for Machine Learning is the cluster runtime with ML libraries and distributed training tools preinstalled.

Data/models exceeding **single-machine** memory cannot train locally	**Memory constraints**	**Distributed** scaling across GPU workers handles any dataset size
Distributed training code is **complex** and difficult to write correctly	**Code complexity**	**Managed** abstractions (DeepSpeed, TorchDistributor, Ray) hide distribution details
Inter-node communication causes **performance** degradation at scale	**Communication overhead**	**Optimized** pipeline parallelism in DeepSpeed reduces communication costs
Single-machine training has **limited** throughput and speed	**Training scalability**	**Multi-GPU** cluster training enables large-scale model development
Manual **GPU** provisioning and cluster configuration is error-prone	**Infrastructure management**	**Integrated** Databricks Runtime ML infrastructure handles resource allocation

databricks runtime ml ↗ ai runtime ↗ ai runtime quickstart ↗ distributed training ↗

DATA SCIENCE & MACHINE LEARNING

3.4 Model Lifecycle with MLflow

Repeat any experiment, and know which version runs in production.

With compute in hand, a team trains many models and compares them, over months and across people.

CHALLENGES

- Each team keeps its metrics to itself.
- Nothing tracks model versions over time.
- Promotions are manual and error-prone.

MLflow keeps the record.

- It records every training run with its parameters, metrics, code version and artifacts, so any result can be **reproduced and compared**.
- Every model in production is **registered in Unity Catalog** with its versions, its aliases such as champion and challenger, and its **lineage back to the training data**.
- Promoting a new version means **moving an alias**, and model update triggers or deployment jobs can start the next steps **automatically**.
- Experiments, models and prompts live in Unity Catalog with the **same permissions as tables**.

Model Lifecycle with MLflow GA

MLflow tracks experiments and manages the lifecycle of models.

- Experiment tracking
- Models in Unity Catalog *model registry*

Experiments are not **reproducible** without tracked parameters and code versions	**Reproducibility**	**Centralized** experiment tracking logs all parameters, metrics, and artifacts
Teams work in **isolation** with scattered metrics and results	**Collaboration**	**Unified** model repository enables comparison and knowledge sharing
No **visibility** into models across workspaces	**Cross-workspace governance**	Unity Catalog integration enables **cross-workspace** model access and discovery
Manual **version** management creates inconsistency and confusion	**Model versioning**	**Standardized** model format ensures compatibility across tools
Agent/LLM behavior is **opaque** with no intermediate step visibility	**Debugging & observability**	MLflow Tracing **records** inputs, outputs, and metadata for each step

DATA SCIENCE & MACHINE LEARNING

3.5 Model Serving

ML models

Any registered model answers requests, in real time or in batch, with no serving stack to run.

A registered version is a record. To be useful it has to answer requests.

CHALLENGES

- Each kind of model needs its own serving system, provisioned by hand, with access control split across them.
- Idle machines keep costing money.

Model Serving is one serverless service for every model type.

- Its endpoints run the registered version, for **real-time requests and for batch scoring**.
- Custom model endpoints serve models built with **scikit-learn, XGBoost, PyTorch or Hugging Face**.
- Endpoints scale with demand, **down to zero**.
- Endpoint telemetry **monitors the usage** of each endpoint.
- Inference tables keep **each request and its response** in a Delta table.

Model Serving *ML models* GA

Model Serving deploys registered models as serverless endpoints.

- Custom model endpoints
- Endpoint telemetry
- Inference tables

multiple serving systems per provider	**Deployment**	**unified** control plane and REST API
manual infrastructure **provisioning**	**Scaling**	**serverless** auto-scaling to demand
fragmented model access control	**Governance**	**centralized** UC/Gateway governance
separate queries for features and models	**Integration**	integrated **feature lookup** in serving
idle compute **wastes** spend	**Cost**	**pay-per-token** with scale-to-zero

model serving ↗ foundation model overview ↗ model serving ↗ model inference ↗

DATA SCIENCE & MACHINE LEARNING

3.6 Deployment Automation

Each change reaches production through the same automated path.

Features, models, jobs and pipelines exist in a development workspace. The last step is moving them to production without a hand on each one.

- Declarative Automation Bundles describe code, jobs, models and their environments as **files in Git**.
- The Databricks CLI deploys them from a terminal or a **CI pipeline**.
- One bundle deploys to **development, staging and production** targets.

Declarative Automation Bundles GA

Declarative Automation Bundles describe jobs, pipelines, models and their environments as YAML.

bundles ↗ bundles settings ↗

*The chain was built to carry more than classical models. An **agent** calls the **same registered, served and deployed models**, and adds what a conversation needs.*

CHAPTER 4

Agent Development

An agent is a program that calls a model, reads company data and takes actions.

Building one on this platform means adding a few layers to the chain that already serves classical models, rather than standing up a second stack.

Business users build agents without code in Genie Agents (see AI for Business Users).

- Model Access
- Custom LLMs
- Batch Inference: AI Functions
- Knowledge Retrieval
- Tools
- Agent Build
- Agent Deployment
- Agent Memory

AGENT DEVELOPMENT

4.1 Model Access

No agent is tied to one model or one provider.

An agent starts with a model. Which one, and from which provider, will change over the life of a project, so the choice must stay cheap to revisit.

CHALLENGES

- Depending on one provider creates lock-in.
- Models from different providers are tested piece by piece.

Foundation Model APIs serve the hosted models through one API.

- The hosted models come from providers that include **Anthropic, OpenAI, Google and Meta**.
- They are billed **per token**, with a priority option, or with **provisioned throughput**.
- The catalog includes OpenJev, the Databricks **judge model**.
- Because applications call the **same API for every model**, moving to a newer model stays a **small change**.
- Models hosted at a provider are registered as **external models**, with their keys stored centrally.
- Unity Gateway is the **single access layer** in front of all of these models.

Foundation Model APIs *hosted models* GA

Foundation Model APIs serve Databricks-hosted models through one API.

- Pay-per-token
- Priority pay-per-token
- Provisioned throughput
- OpenJev *Databricks judge model*

Vendor **lock-in** to single model provider	**Provider choice**	**Multi-provider** unified API (Anthropic, OpenAI, Google, Meta, Qwen, Grok, others)
Unpredictable token **spend** growth	**Cost management**	**Provisioned throughput** (on-demand or reserved) + **priority** mode pricing
Unavailable endpoints during peak usage	**Production reliability**	**SLA-backed** capacity with reserved or on-demand provisioning
Fragmented testing across providers	**Model evaluation**	**Unified endpoint** for A/B testing and model swapping
Expensive pay-per-token for production	**Deployment**	**Flexibility** to move between pricing tiers

AGENT DEVELOPMENT

4.2 Custom LLMs

Tune a model on company data without owning GPUs.

Sometimes no general model knows the company's vocabulary or tasks well enough, and the model has to be adapted.

CHALLENGES

- Fine-tuning needs GPUs that most teams do not have on hand.
- Once tuned, the model often sits apart from the systems that serve the others.

AI Runtime runs the fine-tuning on serverless GPUs.

- It adapts an open model to company data with **LoRA, QLoRA or full fine-tuning**.
- The tuned model is logged in MLflow and served with custom LLM serving on Model Serving, **like any other model**.
- Its **lineage to the training data** stays in Unity Catalog.

Fine-tuning *on AI Runtime* PUBLIC PREVIEW

Fine-tuning runs on AI Runtime with LoRA, QLoRA or full fine-tuning on serverless GPUs.

full fine-tune requires **large GPU memory**	**Memory**	**QLoRA/LoRA** parameter efficiency
fine-tuning **expensive** with full GPU	**Cost**	serverless GPU **pay-per-minute** model
model adaptation **slow** with full training	**Speed**	**low-rank** matrices enable fast updates
one approach for all use cases	**Flexibility**	choose **QLoRA/LoRA/full** per need
fine-tuned models **isolated** from serving	**Integration**	integrate with **MLflow** and **Model Serving**

ai runtime ↗

AGENT DEVELOPMENT

4.3 Batch Inference: AI Functions

Apply AI to large volumes of data and documents in place.

Before any agent, much of the value of a model is in applying it to a whole table at once, with no conversation.

CHALLENGES

- Running a model over company data usually means sending it to an outside service, building separate pipelines for documents and splitting the work into batches by hand.
- What comes back is unstructured.

AI Functions run this work where the information already lives.

- They call a model **from SQL or from a pipeline** to classify, extract or summarize data and documents **at scale**.
- ai_query calls any model, ai_classify classifies, ai_extract extracts fields, ai_parse_document **parses documents** and ai_transcribe **transcribes audio**.
- Classification and extraction also have a **no-code interface**.
- The results come back with a **fixed schema**, and the data **stays in Unity Catalog**.
- Applications call the same functions through a **REST API**.

Batch Inference: AI Functions GA

AI Functions apply a model to table rows from SQL or a pipeline.

- ai_query *general inference*
- ai_classify *classification*
- ai_extract *information extraction*
- ai_parse_document *document parsing*
- ai_transcribe *audio transcription* BETA
- AI Functions REST API

external AI services require data export	**Integration**	**native** functions run inside data platform
model output is **unstructured** for downstream use	**Structure**	**schema-enforced** results support pipelines
manual batch splitting and parallelization	**Scale**	**automatic** parallelization and scaling
data **leaves** workspace for inference	**Governance**	data **stays** in Unity Catalog
build **separate** OCR, NLP, JSON pipelines	**Engineering**	**unified** AI functions in SQL

AGENT DEVELOPMENT

4.4 Knowledge Retrieval

Answers come from the company's own documents and tables, with the source cited.

An agent answers questions as they come, and the quality of its answers depends on what it can find.

CHALLENGES

- Search indexes are built and kept in sync by hand.
- They often match on vectors alone.

AI Search indexes the material the agent needs.

- ai_prep_search first **splits the documents into chunks** and prepares them for it.
- The index covers documents and tables, supports **vector, hybrid and full-text search**, **stays in sync** with the Delta tables, and returns **citations**.
- The ai_search function queries the index **from SQL**.
- Agents can also draw context from **Genie Agents**, the no-code assistants built by domain experts (see AI for Business Users).

AI Search *vector, hybrid and full-text search* GA

AI Search indexes documents and tables for vector, hybrid and full-text retrieval, synced with Delta tables.

manual embedding and index management	**Retrieval**	automatic **embedding** computation + Delta sync
single-mode search (vector only)	**Search**	**hybrid** search (vector + BM25 + rerank)
vector indices **slow to maintain** at scale	**Scale**	auto-sync from **Delta tables**
embeddings **uncontrolled** and opaque	**Governance**	UC **lineage** and **citations** built-in
search **latency** grows with scale	**Performance**	**storage-optimized** endpoints (1B+ vectors)

ai search ↗

AGENT DEVELOPMENT

4.5 Tools

Agents reach tools through one governed route.

Many tasks need actions as well as answers, such as running a query, opening a ticket or changing code, and each tool needs a connection and permissions.

CHALLENGES

- Teams write a custom connector for every one of them and host the servers themselves.
- Credentials pile up across agents.
- Tool use stays hidden.

Managed MCP servers connect the agent to these tools.

- MCP is the **standard way** for an agent to call tools.
- The managed servers expose **Databricks services** such as Genie, AI Search and SQL.
- Managed MCP connectors give governed access to **Google Workspace, Microsoft 365, Slack and GitHub**.
- Teams can register their **own MCP servers** or host them as Databricks Apps, the platform's application hosting (see AI for Business Users).
- Unity Catalog **functions and skills** serve as tools as well.
- Every tool is a **Unity Catalog object**, so permissions and audit work the same way as for data.
- Databricks Sandbox runs the code an agent writes in an **isolated environment**, with scoped access to data.

Managed MCP servers PUBLIC PREVIEW

Managed MCP servers expose Databricks services, such as Genie, AI Search and SQL, to agents.

Custom **connectors** per tool	**Integration**	**Standard** MCP endpoints
Credential **sprawl** across agents	**Security**	**Centralized** auth (OBO)
Agent access **uncontrolled**	**Governance**	**Unity Catalog** enforces grants
Building server **infrastructure**	**Development**	Databricks **operates** it
Agents **infer** context	**Business logic**	Genie **Ontology** grounds queries
Tool usage **hidden**	**Observability**	**Audit** via Unity Gateway

managed mcp ↗ release notes product 2026 september ↗

AGENT DEVELOPMENT

4.6 Agent Build

Agents are written in the framework the team chooses.

Model, retrieval and tools are the parts. The agent itself is the code that decides how to combine them.

- Custom agents carry the **team's own logic**.
- They are written in code with frameworks such as **LangGraph, LangChain or the OpenAI SDK**.
- They can use **any model and any MCP tool**.
- AI Bridge packages **connect these frameworks to Databricks**: databricks-openai, databricks-langchain, databricks-dspy and databricks-mcp.
- The Agent Bricks CLI creates a new project **from templates**.
- Omnigent, a **harness for coding agents** (see Developer Tools), can also run them.

Custom agents *code-first agents, any framework* GA

Custom agents are written in code with any framework and use any model and any MCP tool.

fixed orchestration patterns	**Control**	**custom** agent logic and workflows
predetermined framework choice	**Framework**	**choose** LangGraph, LangChain, OpenAI SDK
limited integrations to external systems	**Tooling**	**MCP servers, UC functions, APIs** available
manual session and memory management	**State**	**managed** memory via Lakebase
external deployment and monitoring tools	**Deployment**	**integrated** governance, tracing, evaluation
fixed model selection	**Model**	**choose** any model per use case

Agent Bricks CLI BETA

The Agent Bricks CLI scaffolds an agent project, runs it locally and deploys it.

- CLI project templates BETA

agent bricks cli ↗

AGENT DEVELOPMENT

4.7 Agent Deployment

A built agent is live for applications to call, with no servers to run.

Once written, an agent has to run in production, and keep running.

- AgentKit, in the databricks-agentbricks package, provides the agent server that wraps the agent loop and **recovers interrupted runs**.
- Agents run on Agent Runtime, hosted on **Databricks Apps**.
- Applications query a deployed agent through the **invocation API**, with a Databricks OAuth token.
- A deployed agent reports its **traces to MLflow** and its **telemetry to Unity Catalog**.

AgentKit *agent library* GA

AgentKit is the Agent Bricks library for agents that run on Agent Runtime: the DurableAgentServer agent server, plus clients for memory, sessions, tracing and authentication.

- databricks-agentbricks *SDK package*

agent server ↗

Agent Runtime *on Databricks Apps* GA

Agent Runtime hosts agents on Databricks Apps.

agent runtime ↗

AGENT DEVELOPMENT

4.8 Agent Memory

Agents remember what users told them.

In production the agent talks to many users over weeks, and what it retains between conversations decides how useful it stays.

CHALLENGES

- Agents often start every conversation from nothing.
- Users then repeat their instructions each time.

Managed memory keeps that knowledge for them.

- An agent searches it when it needs **context from earlier conversations**.
- Sessions keep the **state of each conversation**.
- Memory can also be built with **memory pipelines**.
- Several agents can **share the same memory**.

Managed memory BETA

Managed memory stores facts an agent retains across conversations and searches them semantically.

*Agents are built and running. From here the questions change from building to **operating**: what did the agent do, was the answer good, what did it cost. The next layer makes every agent call **visible and measurable**.*

CHAPTER 5

Observability & Evaluation

An agent in production answers thousands of requests nobody reads.

This layer turns agent quality into a number that can be compared between versions.

It is what allows an organization to put an agent in front of customers and keep it there.

- Tracing with MLflow
- Evaluation with MLflow
- Prompt Registry
- Runtime Telemetry

OBSERVABILITY & EVALUATION

5.1 Tracing with MLflow

Every request an agent handles can be replayed step by step, with its cost.

The first need is to see one request from end to end.

CHALLENGES

- Agent chains are opaque to the people who run them.
- When an answer is wrong, the failing step is hard to isolate.
- The cost of each request is unknown.

MLflow makes every step visible.

- Tracing is turned on with **one call** in the agent code.
- It records **each model call, tool call and retrieval step**, with inputs, outputs, latency and tokens, and it works with the common agent frameworks.
- When an answer is wrong, the trace shows **the step where it went wrong**.
- The traces are **stored in Unity Catalog**.

Tracing with MLflow GA

MLflow Tracing records every request through an agent, step by step.

- Traces in Unity Catalog

agentic chains **opaque** to operators	**Tracing**	**end-to-end** request tracing (OTel, 30+ frameworks)
LLM quality hard to **measure** consistently	**Evaluation**	**LLM-judge** and custom scorer framework
blind production agents and models	**Monitoring**	automatic request/response **logging** with trace context
LLM spend **opaque** per request	**Cost**	**traced tokens** enable cost attribution
failures **hard to isolate** in chains	**Debugging**	granular **intermediate step** visibility

genai ↗ tracing overview ↗ eval monitor ↗

OBSERVABILITY & EVALUATION

5.2 Evaluation with MLflow

Each new version of an agent is measured against the last before it reaches users.

A trace shows one request. Judging quality needs a score across many requests.

CHALLENGES

- Without such scores, traces are reviewed by hand and versions cannot be compared.
- Expert labels sit in separate spreadsheets.

MLflow evaluation measures the answers.

- It applies **LLM judges and custom scorers** to traces.
- It runs in batch before a release and **continuously on live traffic**.
- Each agent version gets its own evaluation run, so **versions can be compared**.
- The Review App lets **domain experts** test the agent in a chat interface and **label its answers**.
- The labels attach to the traces and become **evaluation datasets**.
- They are also used to **align the judges with the experts**.

Evaluation with MLflow GA

MLflow evaluation scores agent answers with judges and scorers, before release and on live traffic.

- LLM judges
- Scorers
- Review App *human labeling*

no **metrics** on response quality	**Quality**	**LLM judges** score each trace
manual review of traces	**Evaluation**	**programmatic** batch evaluation runs
blind to production agent quality	**Production**	**automated** monitoring on live traffic
slow **iteration** without clear signals	**Feedback**	**traces** carry scorer feedback
can't **compare** agent versions	**Comparison**	**evaluation runs** tracked per version

genai ↗ eval monitor ↗ label existing traces ↗

Review App *human labeling* GA

The Review App lets domain experts test an agent and label its answers.

OBSERVABILITY & EVALUATION

5.3 Prompt Registry

`prompt management`

Know which prompt each version of an agent uses.

Scores change when a prompt changes, and prompts change often.

CHALLENGES

- Prompts change with no record.
- Staging and production drift apart.

Prompt Registry keeps a version of every prompt.

- Staging and production **aliases** point to them, so a change is **tracked like a code change**.
- Each prompt is stored with its **model parameters**, such as the temperature.
- Access to each prompt follows the **Unity Catalog permissions** set for experiments and models.

Prompt Registry *prompt management* GA

The Prompt Registry versions prompts with aliases and the model parameters they were tested with.

prompt changes **untracked** over time	**Versioning**	version **history** per prompt
staging/prod prompts **diverge**	**Deployment**	**alias-based** stage control
prompts **scattered** across notebooks	**Reusability**	centralized prompt **registry**
prompt access **uncontrolled**	**Governance**	UC **permissions** on prompts
model outputs **hard to replicate**	**Reproducibility**	frozen prompt **versions** per run

prompt registry ↗

OBSERVABILITY & EVALUATION

5.4 Runtime Telemetry

Endpoints and applications are watched from one place, with the same tools as the rest of the platform.

Traces and scores cover the agent's answers. The endpoints and applications that run it also have to be watched as infrastructure.

- The **endpoint telemetry** of Model Serving is the first source.
- Databricks Apps export their telemetry to Unity Catalog, with **agent traces in MLflow**.
- The **inference tables** of Model Serving are the record used for quality analysis.
- System tables record **tokens, latency and errors** for audit and troubleshooting.

*Everything measured so far was measured **after the fact**. The next layer puts the **rules in place before the call**, so a bad call does not happen.*

CHAPTER 6

AI Governance & Security

AI stays out of production when nobody can say who may use which model, with which data, at what cost, and who answers for it.

This layer applies to models, agents and tools the same rules Unity Catalog already applies to data, and puts one gateway in front of every model call.

It is the layer a security or risk officer will look at first.

- Unity Catalog for AI Assets
- Unity Gateway: Policies
- Unity Gateway: Monitoring
- Unity Gateway: Cost & Capacity
- Unity Gateway: Coding Agent Governance
- Security & Compliance

AI GOVERNANCE & SECURITY

6.1 Unity Catalog for AI Assets

Each model, agent and tool can be traced to the data behind it and to everyone who used it.

Models, agents, tools and traces are assets in their own right, and governing them starts by registering them like tables.

CHALLENGES

- Permissions are enforced differently across models, agents and tools.
- External credentials are scattered.
- Nobody knows which data feeds which model.

Unity Catalog extends the existing rules to these assets.

- These assets include **models, functions, hosted model services, external model provider services, memory stores, skills and traces**.
- Each of them is a Unity Catalog object with **permissions, lineage and an audit trail**.
- Credentials for external services are stored in the catalog and **encrypted**.
- Permissions also cover the use of **foundation models and AI Functions**, and **ABAC GRANT policies** apply to models and services.

Unity Catalog for AI Assets GA

Models, functions, model services, memory stores, skills and traces are securables with the same permission model as tables.

- Models
- Functions
- Model services *hosted model endpoints* BETA
- Model provider services *external providers*
- Memory stores BETA
- Skills BETA
- Traces
- Foundation model permissions
- AI Functions permissions PUBLIC PREVIEW
- ABAC GRANT policies for models and services

AI models, agents, functions **ungoverned** individually	**Asset governance**	UC governs **all asset types**: models, agents, UC functions (tools), MCP services, connections, credentials
Inconsistent **permission** enforcement across assets	**Access control**	**RLAC/ABAC** with role and attribute-based policies

AI GOVERNANCE & SECURITY

6.2 Unity Gateway: Policies

Every model and tool call follows the same policies.

The catalog sets who may use each asset. Each model call and each tool call also needs control at the moment it happens, on what goes in and what comes out.

CHALLENGES

- Prompts and responses go through without guardrails.

Unity Gateway applies the policies on every call.

- **Tool calls** pass through it as well as model calls.
- Service policies set the **rules for the calls** to each service.
- LLM-as-a-judge policies **check prompts and responses** with a model.
- Sensitive data is **redacted**, and partner guardrails can be added.
- Request tags **attribute each call** to a user, team or project.

AI GOVERNANCE & SECURITY

6.3 Unity Gateway: Monitoring

See who calls what, and keep a record of every call.

Policies decide what passes. Each call through the gateway also has to be on record, for audit.

CHALLENGES

- Teams connect to providers on their own, so nobody sees who uses which model.
- Nothing is logged for an audit.

Unity Gateway records the traffic that passes through it.

- Inference tables, as for Model Serving endpoints, log **every request and response** through the gateway.
- Usage is tracked **per user and team**, and a usage dashboard shows it.
- A unified trace table collects the traces of **all gateway traffic** in one place, including the **coding agents** connected through the gateway.

AI GOVERNANCE & SECURITY

6.4 Unity Gateway: Cost & Capacity

Keep AI costs within budget.

Every call through the gateway has a price, and without limits a single team or a runaway agent can spend far more than planned.

- Unity Gateway applies **budgets and cost caps** per user, team or project.
- **Rate limits** cap the number of requests.
- Smart routing **picks the model** for each request and **falls back** to another one when a provider is unavailable.

AI GOVERNANCE & SECURITY

6.5 Unity Gateway: Coding Agent Governance

Developers keep their own coding agents and the platform sets the rules for their model calls.

Developers also call models from their own machines, through coding agents, outside any application the platform hosts, and those calls need the same rules.

- Coding agents connect to Unity Gateway with the **ug CLI**.
- Their configuration is **managed centrally**.
- Their model calls then follow the **same policies, monitoring and cost controls** as the rest of the AI traffic.

AI GOVERNANCE & SECURITY

6.6 Security & Compliance

Databricks AI Security Framework

Known AI risks map to controls the platform provides.

Permissions and guardrails address the risks a team already knows. A security officer also needs the full list, and the control that covers each one.

- The Databricks AI Security Framework lists the **risks** of AI systems and the **controls** that address them.
- The current version adds risks **specific to agents**.
- The controls include **least-privilege tool access** in Unity Catalog, **sandboxed code execution**, guardrails in Unity Gateway, and **human review**.
- Databricks Geos keep data in a **chosen geography**, for data residency.
- The compliance security profile turns on **additional security controls** for regulated workloads.

Security & Compliance *Databricks AI Security Framework* GA

The Databricks AI Security Framework maps AI risks to controls; data residency and compliance profiles apply to AI workloads.

- Databricks Geos *data residency*
- Compliance security profile

blog post ↗

*Everything so far is operated by technical teams. The next layer is how **people who are not technical** reach the same data and agents, in **plain language**.*

CHAPTER 7

AI for Business Users

Business decisions rest on data most decision-makers cannot query themselves.

This layer decides whether the governed data and the agents reach them at all, and whether what they get carries the same permissions and definitions as the reports.

It is where the investment in the layers below becomes visible to the business.

- Genie One
- Genie Agents
- Genie Ontology
- Genie Integrations
- AI/BI Dashboards
- Applications

AI FOR BUSINESS USERS

7.1 Genie One

conversational assistant

Anyone in the business can question the data directly.

The data, the definitions, the models and the agents are in place. The business still needs a way to ask.

CHALLENGES

- Most people who need data do not write SQL.
- Their questions queue up at the data team.

Genie One gives everyone a single entry point to the data.

- Business users get answers from governed company data, with **citations** and with their **own permissions** applied.
- When a dashboard or an agent **already answers** the question, Genie One sends the user there.
- It runs on the **web**, as a **desktop app** on macOS, and on **iOS and Android**.
- Other AI assistants reach it through the **Genie One MCP server**, and **web search** adds outside sources to its answers.
- Users with **account-only access** can use it too.
- Genie budgets set **per-user limits on spend**.

Genie One *conversational assistant* GA

Genie One is the conversational assistant for business users on structured and unstructured data.

- Web app
- Desktop app *macOS* BETA
- iOS app
- Android app
- Genie One MCP server
- Web search BETA
- Account-only user access BETA

Business users **blocked** by technical Databricks interface	**Access**	Simplified, non-technical **front door** with natural language chat
Analyst **dependency** for every data question	**Self-service**	Users **answer** ad-hoc questions directly without SQL or waiting
Hard to find **relevant** dashboards, agents	**Discovery**	Unified search and **routing** to appropriate

AI FOR BUSINESS USERS

7.2 Genie Agents

no-code agents

The knowledge of each business area becomes available to everyone in it.

Some questions need the knowledge of a business area as well as its data: how a region defines a lead, which exceptions apply.

CHALLENGES

- That knowledge stays with a few people.
- Dashboards only answer the questions they were built for.
- Deeper analysis is spread across separate tools.

Genie Agents package that expertise for a business area.

- A domain expert builds one **without code**.
- The assistant can **follow up** on questions, run **multi-step analyses** and **explain its reasoning**, and it can be embedded in other applications.
- Agents are built on the catalog's **tables, metrics and domains**.
- They are **shared across teams** and can be called by custom agents (see Agent Development).

Genie Agents *no-code agents* GA

Genie Agents are no-code assistants built by domain experts on selected tables and metrics.

Dashboards answer only **predefined** questions	**Coverage**	Agent-based **follow-up** and ad-hoc analysis beyond dashboard scope
Metrics defined **inconsistently** across teams and dashboards	**Accuracy**	Curated **semantic context** with shared metric definitions and business rules
Complex "why" and "what-if" analysis **fragmented** across tools	**Reasoning**	Agent mode with **multi-step** investigation, hypotheses, and explainable reports
Business knowledge **trapped** with subject-matter experts	**Expertise**	Domain experts **encode** logic once, shared as reusable agents
Analytics **isolated** from business applications and workflows	**Integration**	Agents **embedded** in external apps, APIs, chatbots, and frameworks
AI answers raise data access and **compliance** concerns	**Governance**	Unity Catalog **permissions** automatically applied per user

genie one ↗ release notes 2026 ↗ genie agents ↗

AI FOR BUSINESS USERS

7.3 Genie Ontology

business context layer

Each metric has one certified definition.

The definitions set in Unity Catalog feed those answers. The remaining risk is which definition Genie picks.

CHALLENGES

- Metrics are defined differently across dashboards, queries and teams.
- Business context is scattered in documents, chats and tickets.
- An AI can then pick an obsolete or draft definition.

Genie Ontology grounds answers in the certified version.

- It points each KPI to its **metric view** in Unity Catalog.
- The ontology also **learns business context** from existing dashboards, queries and pipelines.
- It ranks sources by **certification, usage and freshness**, so Genie answers from **certified definitions first**.

Genie Ontology *business context layer* PUBLIC PREVIEW

Genie Ontology holds business context over metrics and domains so answers use certified definitions.

Metrics defined **differently** across dashboards, queries, and teams	**Consistency**	Canonical **Unity Catalog Metric Views** define each KPI once, governed globally
Genie may **choose** obsolete or draft definitions over trusted sources	**Authority**	OntoRank **scores** sources by certification, usage, and freshness
Business definitions **fragmented** across dashboards, SQL, docs, chat, tickets	**Context**	Unified **business-context layer** combines modeled and learned context
AI answers **lack grounding** in authoritative business definitions	**Trust**	Genie **uses** certified metrics and Pages in answering, with governance applied
Only critical metrics are **formally defined**; long tail of context is scattered	**Coverage**	Ontology **learns** context from existing assets (dashboards, queries, pipelines)
Ontology snippets may **leak** access to data users cannot see	**Permissions**	Context inherits **Unity Catalog permissions** at retrieval time per user

genie ontology ↗ blog post ↗

7.4 Genie Integrations

Business users get answers without leaving the tool they are in.

Many users spend their day in chat tools and spreadsheets and will not open another application to ask a question.

- Genie answers questions inside **Microsoft Teams and Slack**.
- It also works in **Excel and Google Sheets**.
- It is available in **Microsoft Copilot Cowork** as well.
- Answers carry the **same permissions and definitions** as in the workspace.

AI FOR BUSINESS USERS

7.5 AI/BI Dashboards

Analysts and business people serve themselves, without waiting for a specialist.

Some questions come back every day and deserve a fixed view rather than a new conversation.

CHALLENGES

- Building a dashboard takes SQL skills, so requests wait.

AI/BI Dashboards come with an assistant.

- A dashboard can be built from a **plain-language request**.
- The assistant finds the tables, writes the queries and picks the charts, and the author **approves each step**.
- Dashboards use the **semantic definitions** of Unity Catalog.
- Teams **subscribe** to a dashboard in Slack or Microsoft Teams.

AI/BI Dashboards GA

AI/BI Dashboards are built with an assistant and share the catalog's semantic definitions.

Dashboard **authoring slow** with manual SQL and visualization configuration	**Speed**	AI-assisted **workflow** plans steps from table discovery to layout
High **SQL barrier** for dashboard creation	**Accessibility**	Natural language **prompts** ("show revenue by region as bar chart") generate dashboards
Authors struggle to **choose** the right chart type and configuration	**Visualization**	Agent **recommends** and auto-configures appropriate visualization
Joining tables and creating **calculated metrics** is error-prone and slow	**Data prep**	Agent **executes** table joins, aggregations, and metric creation on request
Troubleshooting and **refining** dashboards requires specialist skills	**Iteration**	Agent **identifies** errors and refines interactively with author approval
Self-service BI **blocked** by specialist dependency	**Breadth**	Analysts and business users **move** from question to governed dashboard independently

dashboards ↗ release notes 2026 ↗ dashboards ↗

7.6 Applications

An internal application ships without its own infrastructure, identity system or secrets.

When the same workflow comes back every week, with inputs and decisions, it is worth turning into a small application.

CHALLENGES

- Usually each one runs on separate infrastructure, with its own identity management and secrets.
- It sits apart from the governed data.
- Access control and audit trails are hard to put in place.

Databricks Apps hosts them on the platform.

- Developers build applications with the **frameworks they already use**, such as Streamlit, Gradio, React or FastAPI, and deploy them.
- Databricks AppKit is a **framework** for building them.
- They sign users in with the **platform's identity** and read data through Unity Catalog with **each user's permissions**.
- App Spaces give admins **one place to govern** who deploys and uses which app.
- Genie App Builder generates an app from a **plain-language description**, within the limits the admins set.
- Genie ZeroOps runs **pipeline operations** with an agent that proposes a fix and **waits for human approval**.

Databricks Apps *application hosting* GA

Databricks Apps hosts applications on serverless compute with the platform's identity and permissions.

- Databricks AppKit *app framework*
- App Spaces BETA

Apps require **separate infrastructure** outside Databricks platform	**Infrastructure**	**Serverless Databricks Apps** eliminate external hosting costs and complexity
App-level access control and **audit trails** difficult to implement	**Governance**	App Spaces provide **admin governance** framework for app access and deployment
Apps require **separate identity management** and secrets handling	**Authentication**	Native **OAuth and UC integration** with app vs user-principal auth options
Apps **isolated** from governed data lake	**Data access**	Native **Unity Catalog integration** for

__Developers__ also work through agents, at their own desks, and the platform has to serve them __without a separate set of rules__.

CHAPTER 8

Developer Tools

Developers choose their own coding agents, and the platform has to accept that choice without losing control of the model calls.

This layer is where those agents meet the platform.

- Omnigent

DEVELOPER TOOLS

8.1 Omnigent

coding agent harness

Developers switch between coding agents without re-explaining the task.

Working with several coding agents on one task, each in its own tool, is the common case.

CHALLENGES

- Each coding-agent tool keeps its own context, so content is copied between them by hand.

Omnigent puts them behind one interface.

- It works with coding agents such as **Claude, Codex or Cursor**.
- Agents are defined in **portable YAML**, for local or managed execution.
- Developers can **share a live session**.
- The managed version runs code in **Databricks Sandbox**.
- The agentic code converter **translates SQL** written for other warehouses into Databricks SQL.

Omnigent *coding agent harness* BETA

Omnigent is a harness that runs coding agents such as Claude Code or Codex with shared sessions and central policies.

each agent harness **isolated** ecosystem	**Fragmentation**	**unified** interface across harnesses
rewrite code to change agents	**Switching**	**common API** switches between harnesses
manual copying of content between tools	**Context**	**automatic** context sharing
no centralized control mechanism	**Governance**	**contextual policies** at meta layer
agents operate in **isolation**	**Collaboration**	**live session sharing** for teamwork

omnigent.ai ↗ omnigent ↗

Agentic Code Converter *SQL migration* GA

The agentic code converter uses Genie Code to translate SQL from other warehouses into Databricks SQL or notebooks.

agentic code converter ↗

*The same stack also carries **complete products**.*

CHAPTER 9

Vertical Applications

Databricks builds complete applications for specific domains on the same platform, with the same catalog, agents and governance a customer would use.

They are the test of the platform: if the layers fit, a product can be built on them without a second stack.

- CustomerLake
- Lakewatch

VERTICAL APPLICATIONS

9.1 CustomerLake

customer data platform

Campaigns and personalization run on governed customer profiles that stay current.

The first is for customer data, which every business function reads and writes.

CHALLENGES

- Each system holds its own part of the customer, often duplicated and without lineage.
- Audiences are built and activated by hand.

CustomerLake covers this work as a ready-made application.

- It joins transactions, behavior and engagement into **one profile per customer**, on the lakehouse.
- AI-assisted **identity resolution** merges duplicate records.
- Campaign agents **create and activate audiences**, and a real-time profile API feeds **personalization**.

CustomerLake *customer data platform* PRIVATE PREVIEW

CustomerLake is a customer data platform on the lakehouse, with identity resolution and campaign agents.

Customer data **fragmented** across systems	**Customer insight**	**Customer 360** profiles unifying transactions, behavior, engagement
Duplicate customer records create **fragmentation**	**Identity resolution**	**AI-assisted** identity matching (deterministic, probabilistic, ML-based)
Manual audience segmentation and activation	**Campaign automation**	**Campaign agents** for audience creation, activation, and optimization
Customer data **ungoverned** with no lineage	**Governance**	Embedded in lakehouse with **Unity Catalog** governance and audit
Batch profile lookups delay personalization	**Real-time personalization**	**Real-time profile API** with predictive models for instant attributes

blog post ↗ press release ↗

VERTICAL APPLICATIONS

9.2 Lakewatch

agentic SIEM

Security logs stay on the lakehouse and agents investigate the alerts.

CustomerLake applies the platform to customer data. The same pattern applies to security data, where volume is the problem.

CHALLENGES

- Telemetry is split across separate tools.
- Threat hunting and triage are manual.
- Detection rules are hard to version and test.

Lakewatch runs security operations on the lakehouse.

- It keeps security, IT and business logs in one place at **object-storage cost**, so long retention is affordable.
- Autonomous agents **investigate alerts** and help write **detection rules**, which are versioned and tested like code.
- Panther, whose acquisition Databricks has announced, brings its **detection engine** to Lakewatch.

Lakewatch *agentic SIEM* PRIVATE PREVIEW

Lakewatch is an agentic SIEM that keeps security logs on the lakehouse and investigates alerts with agents.

Security telemetry **siloed** in separate SIEM vendors	**Data unification**	**Unified** security, IT, and business data on lakehouse with UC governance
Manual threat hunting and incident triage	**AI-driven automation**	**Autonomous agents** for investigation, rule authoring, and data normalization
Logs **deleted** for cost; limited retention capacity	**Data scale**	**Unlimited** petabyte-scale ingestion and retention (object storage economics)
Detections **scattered** across tools; hard to version and test	**Detection management**	**Detection-as-Code** with version control and automated testing
Slow log searches across large datasets	**Query performance**	**Sub-second** queries across billions of records for rapid investigations
Heterogeneous log formats from diverse sources	**Log normalization**	**Automatic OCSF** mapping (Open Cybersecurity Schema Framework)

blog post ↗ press release ↗

Closing

The result is open tables, one catalog that governs them, models and agents built on both, every call measured and controlled, and business users served in plain language.

Each new layer reuses the catalog, the compute and the serving layer beneath it, so adopting agents does not mean rebuilding the data platform.

The data stays in the organization's own storage, in open formats, and the choice of models stays open.

Capability map

1. DATA STORAGE & MOVEMENT

Table Formats
- Delta Lake
- Apache Iceberg
- Multi-table transactions

Ingestion & Pipelines
- Lakeflow Connect
- Zerobus Ingest
- Spark Declarative Pipelines
- Lakeflow Designer
- Lakeflow Jobs

Transactional & Real-Time Data
- Lakebase
- Lakehouse//RT BETA

Table Maintenance
- Predictive Optimization

2. UNITY CATALOG

Catalog & Access
- Catalog
- Discover
- Lineage
- Role-based access control
- Attribute-based access control
- Access requests PUBLIC PREVIEW
- External engine access
- Secrets

Policy & Classification
- Data classification
- Governed tags
- Governance Hub BETA

Observability
- Data Quality Monitoring

Cost & Usage
- System tables

Semantic Definitions
- Metrics
- Domains
- Pages BETA

Data Sharing
- OpenSharing
- Clean Rooms
- Marketplace

3. DATA SCIENCE & MACHINE LEARNING

Workspace
- Notebooks
- Genie Code

Feature Engineering in Unity Catalog
- Feature tables
- Feature Views
- Online Feature Stores
- Feature serving

Training Compute
- Databricks Runtime for Machine Learning
- AI Runtime PUBLIC PREVIEW

Model Lifecycle with MLflow
- Experiment tracking
- Models in Unity Catalog

Model Serving
- Custom model endpoints
- Endpoint telemetry
- Inference tables

Deployment Automation
- Declarative Automation Bundles
- Databricks CLI

4. AGENT DEVELOPMENT

Model Access
- Foundation Model APIs
- External model providers
- Unity Gateway

Custom LLMs PUBLIC PREVIEW
- Fine-tuning PUBLIC PREVIEW
- Custom LLM serving BETA

Batch Inference: AI Functions
- ai_query
- ai_classify
- ai_extract
- ai_parse_document
- ai_transcribe BETA
- AI Functions REST API

Knowledge Retrieval
- ai_prep_search BETA
- AI Search
- ai_search
- Genie Agents as a context source

Tools
- Managed MCP servers PUBLIC PREVIEW
- Managed MCP connectors BETA
- External MCP servers
- Unity Catalog functions
- Skills BETA
- Databricks Sandbox BETA

Agent Build
- Custom agents
- AI Bridge
- Agent Bricks CLI BETA
- Omnigent as meta-harness

Agent Deployment
- AgentKit
- Agent Runtime
- Agent query interfaces

Agent Memory BETA
- Managed memory BETA
- Sessions BETA
- Memory pipelines BETA

5. OBSERVABILITY & EVALUATION

Tracing with MLflow
- Traces in Unity Catalog

Evaluation with MLflow
- LLM judges
- Scorers
- Review App

Prompt Registry

Runtime Telemetry
- Endpoint telemetry
- Databricks Apps telemetry
- Inference tables
- System tables

6. AI GOVERNANCE & SECURITY

Unity Catalog for AI Assets
- Models
- Functions
- Model services BETA
- Model provider services
- Memory stores BETA
- Skills BETA
- Traces
- Foundation model permissions
- AI Functions permissions PUBLIC PREVIEW
- ABAC GRANT policies for models and services

Unity Gateway: Policies
- Service policies
- LLM-as-a-judge policies BETA
- Sensitive data redaction
- Partner guardrails BETA
- Request tags BETA

Unity Gateway: Monitoring
- Inference tables
- Usage tracking
- Usage dashboard
- Unified trace table BETA

Unity Gateway: Cost & Capacity
- Budgets
- Cost caps
- Rate limits
- Smart routing

Unity Gateway: Coding Agent Governance
- ug CLI
- Centralized coding agent configuration

Security & Compliance
- Databricks Geos
- Compliance security profile

7. AI FOR BUSINESS USERS

Genie One

- Web app
- Desktop app BETA
- iOS app
- Android app
- Genie One MCP server
- Web search BETA
- Account-only user access BETA

Genie Agents

Genie Ontology PUBLIC PREVIEW

Genie Integrations

- Microsoft Teams PUBLIC PREVIEW
- Slack PUBLIC PREVIEW
- Excel
- Google Sheets
- Microsoft Copilot Cowork BETA

AI/BI Dashboards

Applications

- Databricks Apps
- Genie App Builder BETA

Genie ZeroOps PRIVATE PREVIEW

Genie Budgets

8. DEVELOPER TOOLS

Omnigent BETA

Agentic Code Converter

9. VERTICAL APPLICATIONS

CustomerLake PRIVATE PREVIEW

Lakewatch PRIVATE PREVIEW